

CODI Architecture, Training, and Inference

From tokens and transformer blocks to continuous thoughts, self-distillation, answer generation, and our 28-direction state-12 finding.

Goal: make every arrow in CODI understandable. This guide begins with token embeddings, opens one transformer block, follows both training routes, traces exact inference, and finally connects the architecture to our causal PCA and efficiency experiments.

Object	Released GPT-2 CODI setting used in our work
Backbone	GPT-2 small: width 768, 12 blocks, 12 attention heads
Continuous reasoning	Six recurrent latent positions, each reusing the same backbone
Training	Shared teacher/student weights; teacher CE + student CE + endpoint hidden KD
Answer interface	Final In_f state at the colon in 'The answer is:'
Our main finding	PC 4-31: 28 directions preserve 87.8% of baseline exact-match accuracy
Engineering point	PC 0-31: rank 32 preserves 94.4%; LM-head microbenchmark is 11.3x faster

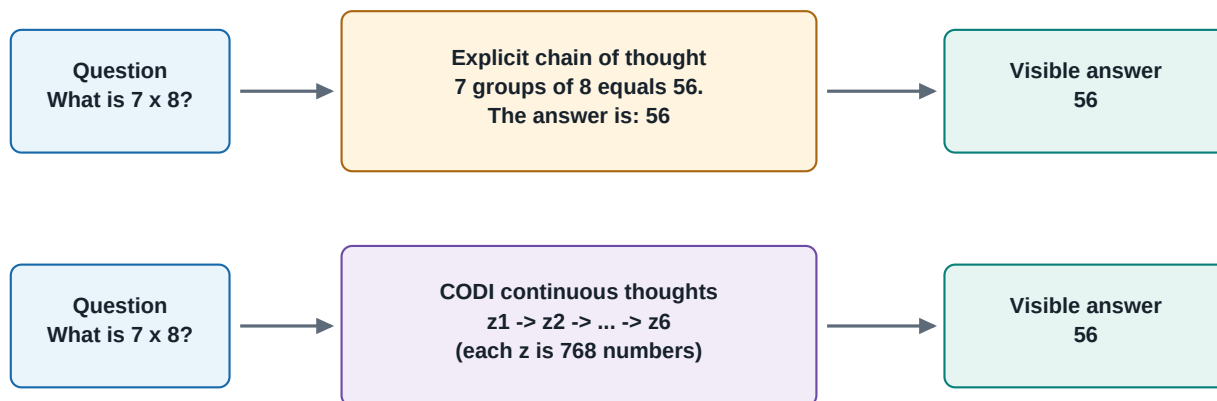
Reading path

Part I builds the machinery. Part II explains self-distillation training. Part III follows inference exactly. Part IV reconstructs our experiments and states what they do and do not establish.

1. The problem CODI is trying to solve

Language models normally reason by producing a sequence of text tokens. Text is interpretable, but it is also a narrow communication channel: every intermediate idea must be squeezed into a vocabulary item and then processed again. CODI asks whether intermediate reasoning can instead live directly in the model's continuous hidden space.

TWO WAYS TO REASON



Training uses explicit reasoning as a teacher signal; inference uses the compact student route.

Figure 1. Explicit chain of thought exposes intermediate words. CODI's student route replaces them with a fixed number of learned vectors.

The central compromise

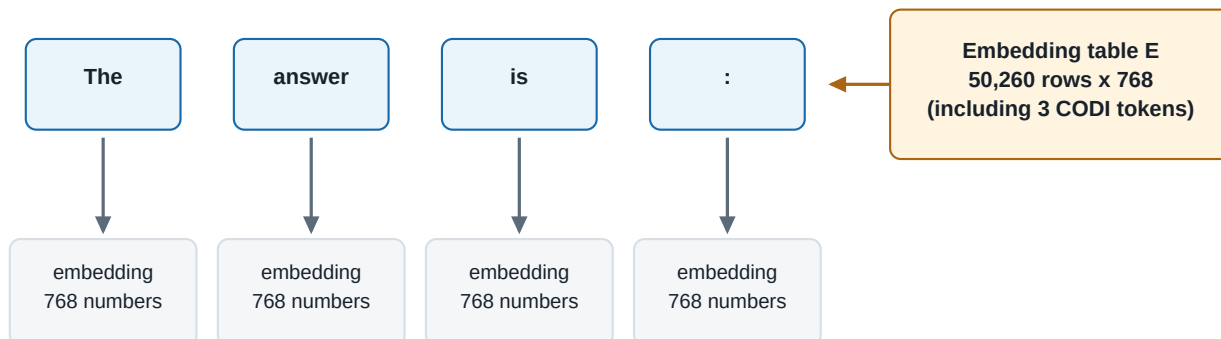
- At training time, keep an explicit chain-of-thought route so the model has a rich teacher signal.
- At inference time, use only the compact student route so reasoning needs a small fixed number of continuous positions.
- Make both routes meet at the same semantic endpoint: the colon immediately before the final answer.

CODI does not remove reasoning. It changes the representation of intermediate reasoning: from visible token IDs to dense 768-number vectors.

2. Tokens, embeddings, and positions

A Transformer does not receive words directly. A tokenizer maps text fragments to integer token IDs. An embedding table then converts every ID into a dense vector. In GPT-2 small, that vector has 768 coordinates.

WORDS BECOME VECTORS



A token ID is only an integer address. The embedding lookup replaces it with a learned 768-number vector.

Positions also receive learned position embeddings so the model can distinguish first, second, and later locations.

Continuous thoughts skip the vocabulary lookup: CODI feeds a learned 768-number vector directly as `inputs_embeds`.

Three CODI-specific IDs

The released model starts from GPT-2's 50,257-token vocabulary and appends PAD, BOT, and EOT. PAD aligns unequal sequence lengths in a batch. BOT marks the beginning of latent reasoning. EOT marks its end.

ordinary GPT-2 IDs: 0 ... 50,256

PAD = 50,257 BOT = 50,258 EOT = 50,259

A position embedding is added because attention alone does not know order. The resulting vector is state 0 in the hidden-state tuple used by our repository.

3. Inside one GPT-2 transformer block

Each block contains two computational sublayers. Masked self-attention moves information between sequence positions. The MLP transforms features within each position. Residual connections preserve the existing representation while adding each sublayer's update.

INSIDE ONE GPT-2 TRANSFORMER BLOCK

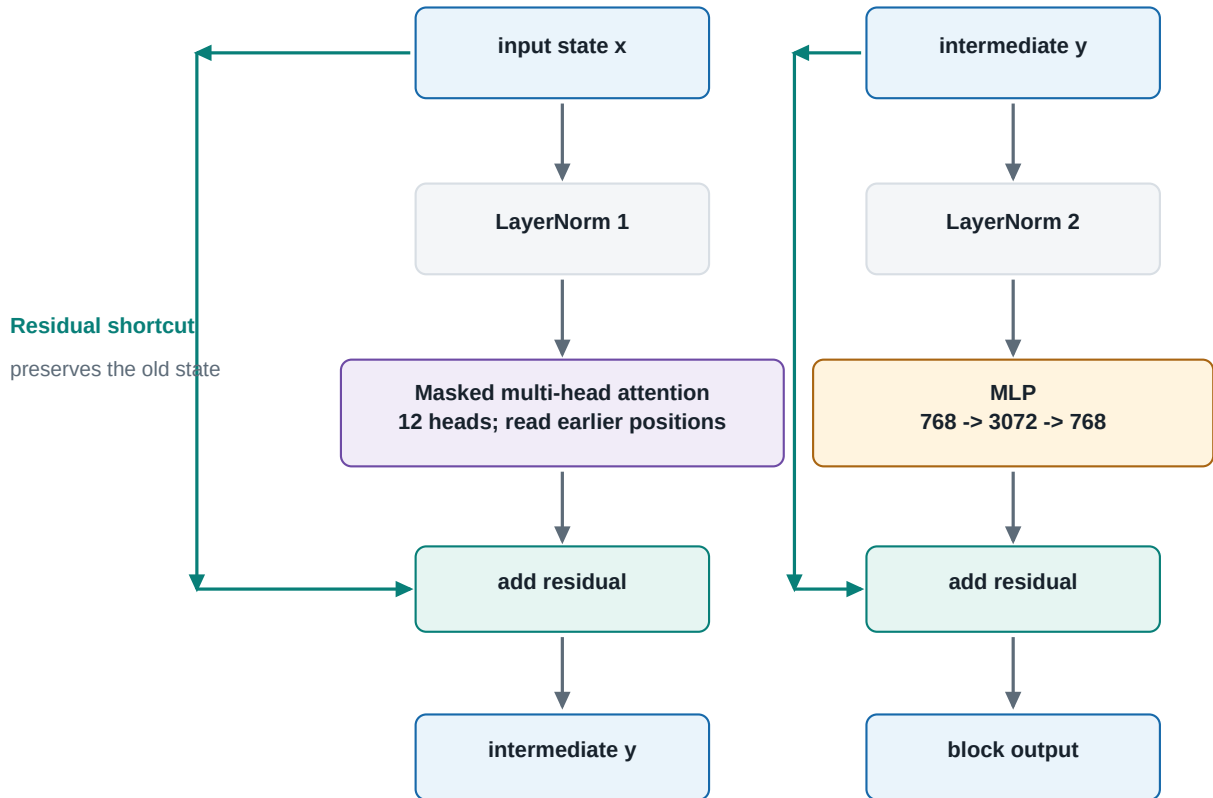


Figure 2. GPT-2 uses a pre-LayerNorm layout: normalize before attention and before the MLP, then add the sublayer result through a residual path.

Dimensions in GPT-2 small

The model width is 768. Attention splits it into 12 heads of 64 coordinates each. The MLP expands 768 to 3,072, applies a nonlinearity, then compresses back to 768. Every block therefore returns the same width it received.

4. Masked self-attention from first principles

At a new position, the model asks: which earlier positions contain information useful for updating me? It forms a query vector for the current position and key/value vectors for stored positions.

A TOY VIEW OF MASKED ATTENTION



For each position, attention creates a query. Earlier positions supply keys and values.

query-key dot products become weights; the weighted sum of value vectors brings useful context forward.

The causal mask forbids looking rightward into future tokens. This makes next-token generation valid.

The essential equations

```
score(i,j) = query_i dot key_j / sqrt(head_dim)
weights_i = softmax(masked_scores_i)
attention_i = sum_j weights(i,j) value_j
```

Twelve heads run in parallel. One head might track numbers, another syntax, another the answer cue. Their outputs are concatenated and projected back to width 768. Head interpretations are examples, not guaranteed one-head-one-concept roles.

5. The full 12-block backbone and state numbering

The same GPT-2 tower processes ordinary tokens, continuous thoughts, the answer cue, and generated answer tokens. It is reused at every new sequence position.

THE COMPLETE 12-BLOCK BACKBONE

Repository naming

state 0 = embeddings

state 1 = after block 0

...

state 12 = after block 11 + In_f

Important

State 12 is not block 12.

There are exactly 12 blocks:
indices 0 through 11.

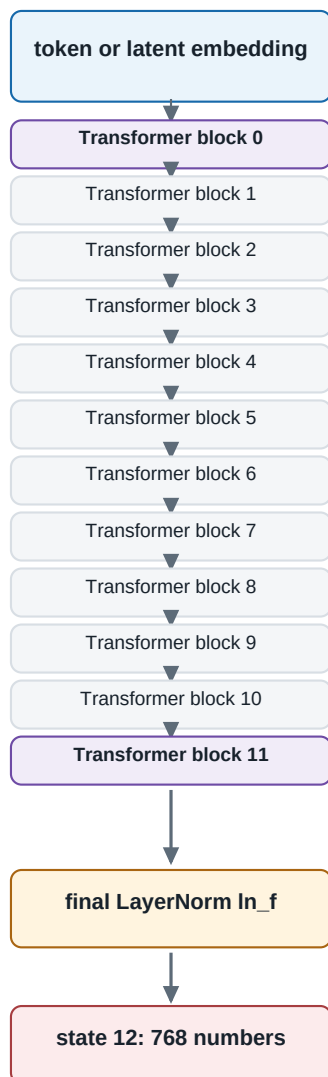


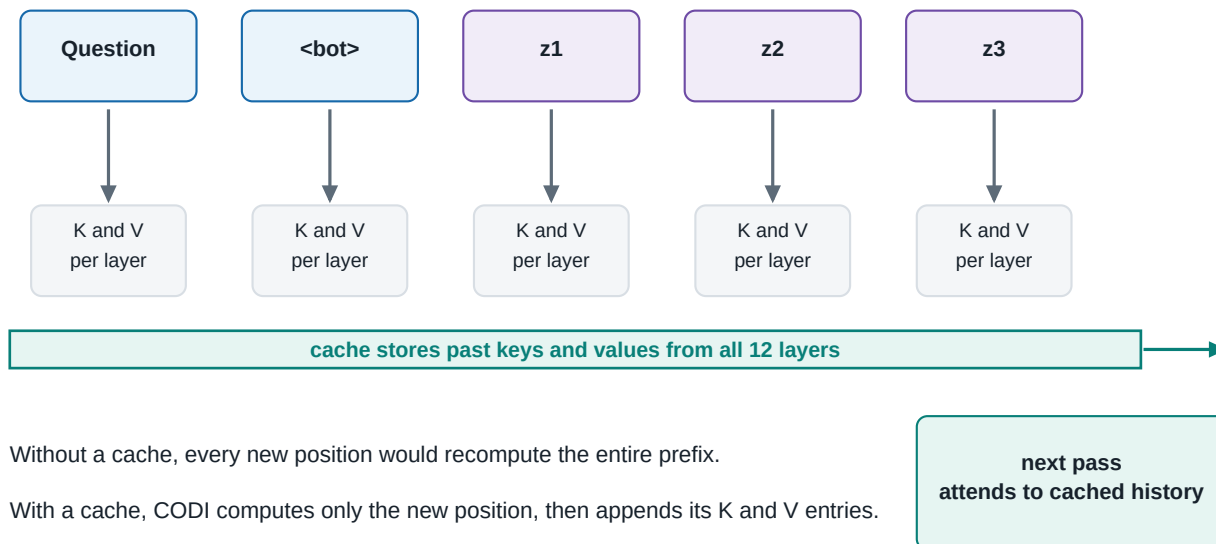
Figure 3. The red state is the exact location of our state-12 answer-interface experiment.

After block 11, GPT-2 applies one final LayerNorm named In_f. The resulting normalized vector is fed to the language-model head. Calling it state 12 describes its position in the 13-entry hidden-state tuple; it does not add another Transformer block.

6. Autoregression and the KV cache

Generation happens one position at a time. Recomputing every old token through all 12 blocks at every step would waste work. The KV cache stores the keys and values already computed for each layer.

WHY THE KV CACHE MATTERS



Conceptual shape per layer: 2 tensors x 12 heads x sequence length T x 64 values per head.

What the cache means for CODI

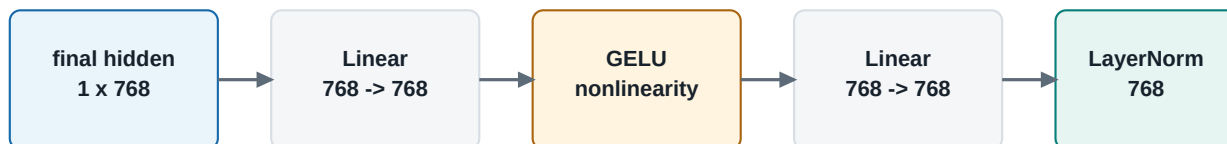
A latent thought is not merely a vector passed forward at the surface. When it travels through the 12 blocks, it contributes a key and value at every layer. Later latent thoughts and answer tokens can attend to those stored entries. That is how information from the silent workspace remains available.

The cache is the model's read-only history for future positions. It avoids recomputation and provides the causal channel through which earlier thoughts influence later ones.

7. The continuous-thought projector

The backbone's last hidden vector is suitable for vocabulary scoring, but CODI also needs a vector to use as the next latent input. A learned projector performs this conversion.

THE LEARNED CONTINUOUS-THOUGHT PROJECTOR



Output: next continuous-thought embedding z

This module is trained. It is not PCA, and it is not the LM head.

The projector converts the backbone's current summary into the vector that will occupy the next latent position.

Because the next vector is fed through all 12 blocks, it can affect later K/V entries and later thoughts.

Released GPT-2 CODI uses six recurrent latent positions during its standard inference path.

One recurrence

```
h_t = GPT2(z_t, past_key_values)
z_(t+1) = LayerNorm(Linear2(GELU(Linear1(h_t))))
```

The output $z_{(t+1)}$ goes directly to GPT-2 as an input embedding at the next position. There is no nearest-word lookup. Six repetitions create z_1 through z_6 while the cache grows.

Do not confuse this projector with PCA. The projector is part of CODI, learned during training, nonlinear, and full-width. PCA is our later offline analysis of many endpoint states.

8. The complete CODI inference architecture

END-TO-END CODI INFERENCE

The 12-block transformer is reused at every new latent and answer position; it is not six separate models.

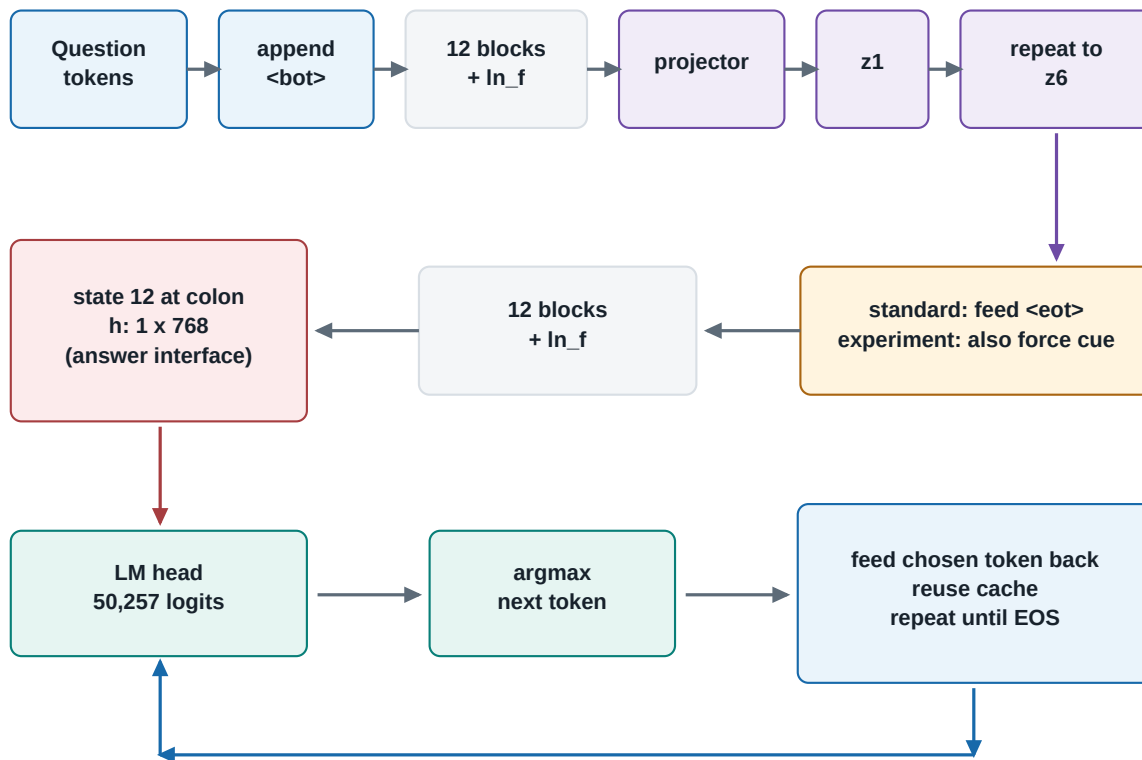


Figure 4. Released student route, with the forced-cue experimental branch labeled explicitly. Every box labeled '12 blocks' is the same weight-shared GPT-2 model used at a new position.

Information flows in two forms

- The current position carries one 768-number vector through the tower.
- The past supplies cached keys and values at all 12 layers.
- The projector creates the next silent input during the latent loop.
- After EOT and the answer cue, the LM head converts the final state into vocabulary scores.

Standard inference uses only the student route. No explicit chain-of-thought teacher trace is supplied at test time.

9. Training overview: one model, two routes

CODI is trained by self-distillation. The teacher and student are roles played by the same underlying language model, not two separately stored backbones. They receive different contexts and therefore produce different hidden trajectories.

ONE MODEL, TWO TRAINING ROUTES

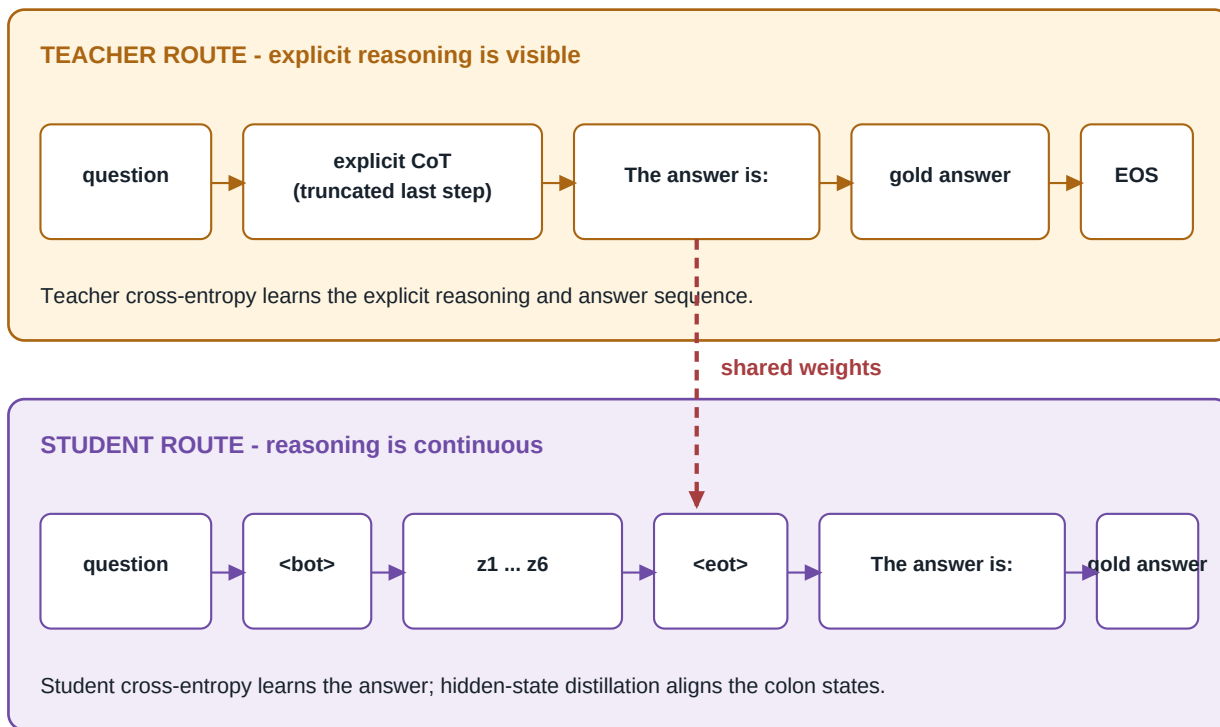


Figure 5. Shared weights receive gradients from three objectives. Only the teacher's hidden target is stop-gradient inside the distillation term.

Why share weights?

A static separate teacher would require another pretrained model and could drift away from the improving student. Joint self-distillation lets the explicit and continuous behaviors develop together while avoiding doubled backbone memory.

10. How one training example is constructed

TOY TRAINING EXAMPLE

Question: A box has 3 rows of 4 balls. How many balls?

Teacher input

Question | 3 rows times 4 balls gives 12. | The answer is: | 12 | EOS

The last explicit equation-like step is removed when formatting the official training trace.

Student input

Question | <bot> | z1 | z2 | z3 | z4 | z5 | z6 | <eot> | The answer is: | 12

The z positions are generated online by the current model; they are not stored text labels.

Both routes must arrive at compatible representations at the colon before 12.

The teacher shows how an explicit reasoner reaches the answer; the student learns a compact internal route.

The anti-shortcut rule

The official formatting removes the final whitespace-separated equation token from the explicit CoT before the answer prompt. The purpose is to prevent the teacher route from placing a trivially copyable final computation immediately next to the answer.

Padding and tokenization

Question, truncated CoT, and answer segments are tokenized with the released sequence contract. Student questions are left padded; teacher sequences are right padded. Exact formatting matters because different whitespace, cue tokenization, or position IDs can change the hidden state even when the visible sentence looks identical.

11. The teacher route

The teacher receives the question, explicit reasoning trace, answer cue, gold answer, and EOS. Teacher forcing means the correct preceding token is supplied at every position, even if the model would have predicted something else.

teacher sequence = question + truncated CoT + 'The answer is: N' + EOS

L_teacher = average negative log probability of each target token

What this objective teaches

- Language modeling over the explicit reasoning trace.
- How the answer cue follows a worked solution.
- How the numeric answer follows the colon.
- A rich internal colon representation that can serve as the student's target.

Toy probability calculation

Suppose the gold answer token is '12' and the teacher assigns it probability 0.80. Its contribution is $-\log(0.80)$, about 0.223. If probability were 0.10, the penalty would be about 2.303. Cross-entropy strongly penalizes low probability on the correct token.

Teacher forcing is a training method. At inference, the model must consume its own chosen tokens instead.

12. The student route

The student starts from question + BOT. It performs a normal cached forward pass, projects the last hidden state, and then generates six continuous thoughts one at a time. After the sixth pass, EOT and the answer cue are appended, followed by the gold answer during training.

```
prefix = question + BOT
z1 = projector(last_hidden(prefix))
for t in 1..6:
    h_t, cache = GPT2(inputs_embeds=z_t, cache)
    z_(t+1) = projector(h_t)
answer_context = cache + EOT + 'The answer is:' + gold_answer
```

The student answer loss

Student cross-entropy directly rewards assigning high probability to the gold answer. It prevents the student from merely imitating a teacher representation without actually producing the correct output.

Why six thoughts?

The paper's sweep found that performance peaked around six: too few positions can limit representational capacity; too many add sequential work and make optimization harder. Our later inference sweep found that this checkpoint could use five without observed accuracy loss, but four caused a clear drop.

13. Where teacher and student meet: the colon

Both routes contain the text 'The answer is:'. CODI extracts the hidden representation at the final colon and aligns teacher and student hidden states across the full hidden-state tuple.

ENDPOINT HIDDEN-STATE DISTILLATION

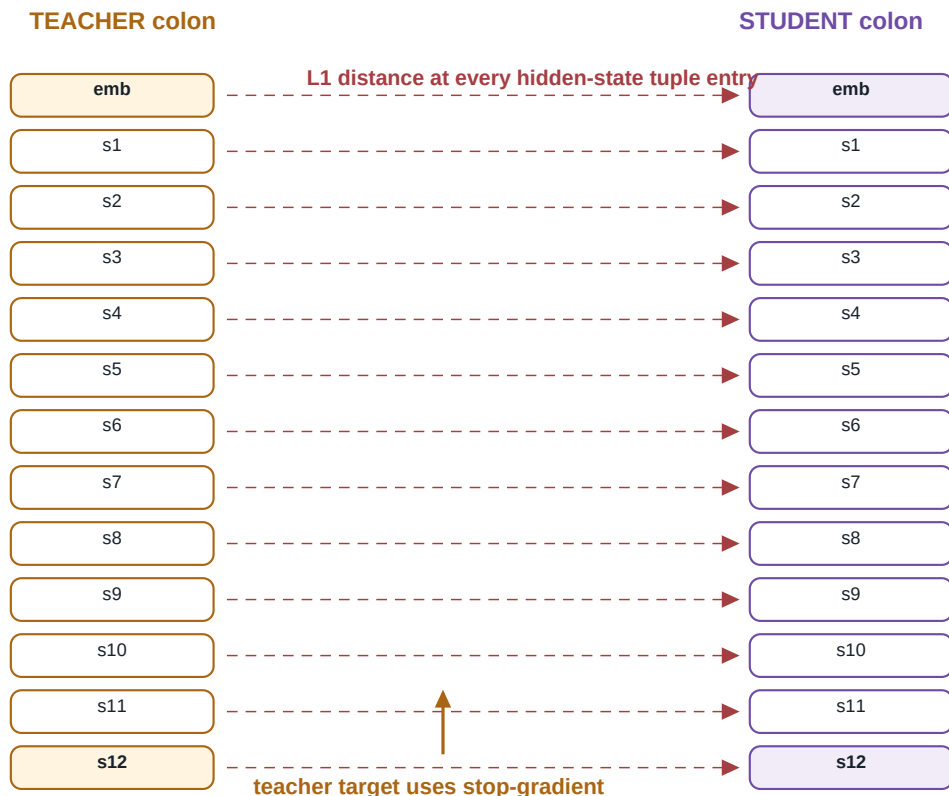


Figure 6. The paper/released implementation matches the endpoint across embedding state plus 12 block outputs. Our repository carefully distinguishes this from block-only diagnostics.

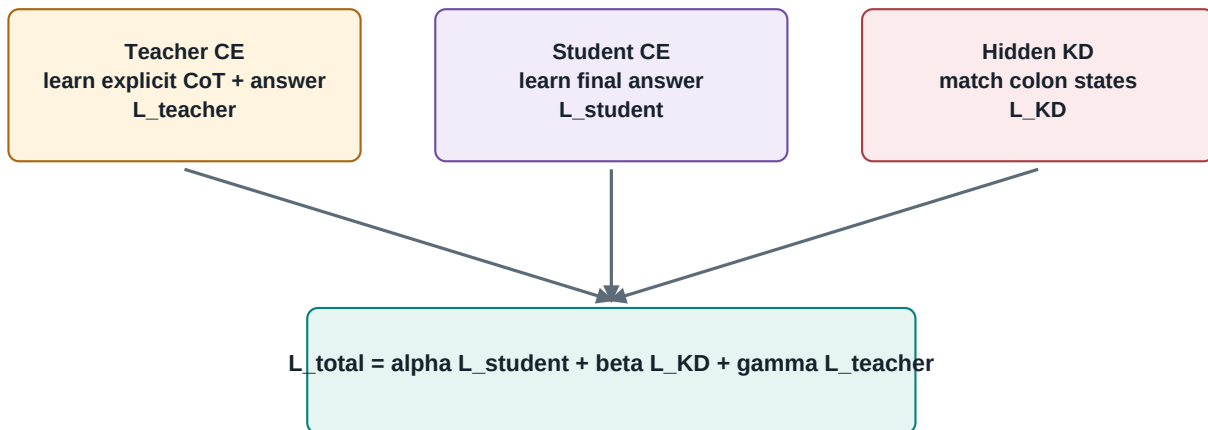
Normalized L1 matching

L_{KD} = average over states and coordinates of $|student - stopgrad(teacher)|$
 with per-state scaling by the teacher batch standard deviation

Scaling prevents a layer with numerically large activations from dominating merely because of units. Stop-gradient makes the teacher representation a target in the KD term; gradients do not pull the target toward the student through that branch.

14. The complete training objective

THE THREE-PART OBJECTIVE



Released GPT-2 setting: $\alpha = 1$, $\beta = 1$, $\gamma = 1$.

Teacher hidden states are detached inside KD, so the target does not move merely to meet the student.

Why all three parts are needed

- Teacher CE preserves an explicit reasoner and continuously improves the teacher role.
- Student CE ties silent computation to the actual answer, not only to representation similarity.
- Hidden KD transfers endpoint organization from the explicit route into the compact route.

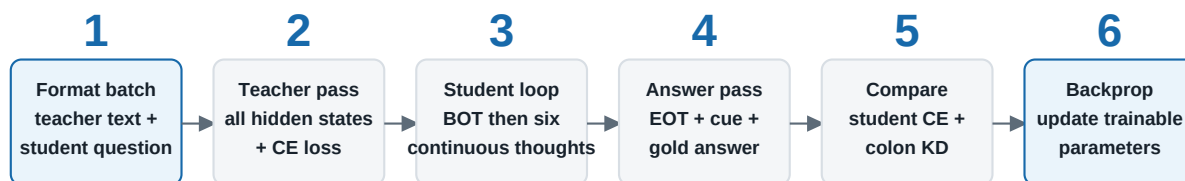
Evidence from the CODI paper

For GPT-2 on GSM8K, the paper reports 43.7% for full CODI. Removing L1 hidden-state distillation drops to 24.5%; a separate static teacher reaches 27.1%; a multitask student without the same self-distillation structure reaches 42.2%; retaining the final CoT step that creates a shortcut reaches 31.7%. These are paper ablations, not our new experiments.

Self-distillation is not a label saying 'think correctly.' It is a geometric training pressure: make the compact route arrive at a teacher-like answer interface.

15. One full training step

ONE OPTIMIZATION STEP



Dynamic student loop

z_1 depends on the current weights; z_2 depends on z_1 ; therefore the six latent positions are generated inside the step.

Memory-saving design

Teacher and student share one backbone. CODI does not require a separately stored frozen teacher model.

Optimization

Paper setting: AdamW, cosine schedule, 3% warmup, effective batch 128, bfloat16, LoRA rank 128.

The public GPT-2 model was trained for 40 epochs with learning rate $3e-3$ (reported about 36 hours on one A100 80GB).

What is updated

The released GPT-2 checkpoint uses LoRA adapters, the learned projector, and the expanded embedding/output table within one jointly optimized computation. The base GPT-2 architecture remains the 12-block causal decoder.

Training data

The paper reports augmented GSM8K training sets with hundreds of thousands of examples. The compact version averages about 20.3 CoT tokens; the verbose natural-language version averages about 49. CODI compresses either into a fixed six-thought student route.

16. LoRA and the trainable parameterization

WHAT LORA CHANGES



In released CODI GPT-2, adapters target attention/MLP modules named `c_attn`, `c_proj`, and `c_fc`.

The latent projector is also learned. New PAD, BOT, and EOT token rows are added to the embedding table.

LoRA rank 128 is a training parameterization. It is unrelated to the 28 PCA directions found later.

Likewise, CODI's projector is a nonlinear 768-to-768 module; our rank-32 readout is a separate inference proposal.

Why low-rank appears twice in this project

LoRA reduces the number of trainable parameter updates inside the Transformer. Our later low-rank LM head reduces inference arithmetic at the output. They use different matrices, different ranks, and different purposes.

Object	When	Rank	Purpose
LoRA adapters	CODI training and inference	128	Adapt attention and MLP weights
PCA band	Our offline analysis	28 scientific / 32 engineering	Describe answer-interface geometry
Factorized LM head	Proposed optimized readout	32	Reduce vocabulary-scoring MACs

17. Exact inference, step by step

Training is over. The checkpoint is put in evaluation mode and weights are frozen. For a batch of questions, the released generation adapter performs the following sequence.

Step	Operation	Information added
1	Normalize and tokenize questions; left pad	Question content and positions
2	Append BOT and run GPT-2 with cache	Initial latent summary
3	Project last hidden state	First continuous-thought embedding
4	Repeat six cached latent passes	Six new K/V entries per layer
5	Feed EOT; forced-cue protocol also inserts cue	Switch from reasoning to answer mode
6	Greedy argmax; at forced colon, read first answer	Next visible token
7	Feed selected token back and reuse cache	Next answer-token context
8	Stop at EOS or maximum length	Final decoded answer

Forced cue in our experiments

For protocol stability, our endpoint experiments manually inserted EOT and the tokenized answer cue after the six thoughts. The state at the cue's colon was captured from the real generation path. This avoids waiting for the model to spell the cue differently across examples.

In ordinary released generation, only EOT is fed at this boundary and visible output is then decoded autoregressively; the model can generate the cue itself. The forced-cue branch is an experimental control, not a new training component.

The first answer token is chosen from the colon state's logits. Later answer tokens have new states because each chosen token is fed back through the model.

18. The LM head in detail

The language-model head is the final linear map that turns a 768-number hidden state into one score for every vocabulary token. It contains no reasoning loop and no attention. It is a bank of vocabulary detectors.

HOW STATE 12 BECOMES A TOKEN



' 12 '		score 8.7
' 11 '		score 7.9
' twelve '		score 6.8
' apple '		score -1.2

argmax selects the largest score. **Softmax** would turn all scores into probabilities, but **greedy decoding** only needs their ordering.

What one detector does

Row w_t is a 768-number pattern associated with token t . The dot product $h \cdot w_t$ is large when the hidden state aligns with that pattern. Computing all dot products together produces the 50,257 logits.

```
z_t = sum over j=1..768 of h_j * W[t,j]
predicted token = argmax_t z_t
```

A logit is an unnormalized score, not a probability. Adding the same constant to every logit changes no argmax. Softmax is only needed when probabilities or sampling are required.

19. Is W already stored in the model?

THE SAME VOCABULARY GEOMETRY IS USED IN BOTH DIRECTIONS



The row used to represent token t on input is also used as the detector for token t on output.

The matrix is learned before inference and stored in the checkpoint. It is not recomputed for every question.

Official GPT-2 vocabulary has 50,257 ordinary rows. CODI adds PAD, BOT, and EOT, but released generation scores only rows before EOT.

Our low-rank factor $A = WU$ is an additional offline artifact; it is not part of the original CODI checkpoint.

Yes - W is learned beforehand

At inference, the model loads W from the checkpoint. Every new hidden state is multiplied by the same stored matrix. GPT-2 ties this output matrix to the input embedding table, which saves parameters and gives tokens a shared input/output geometry.

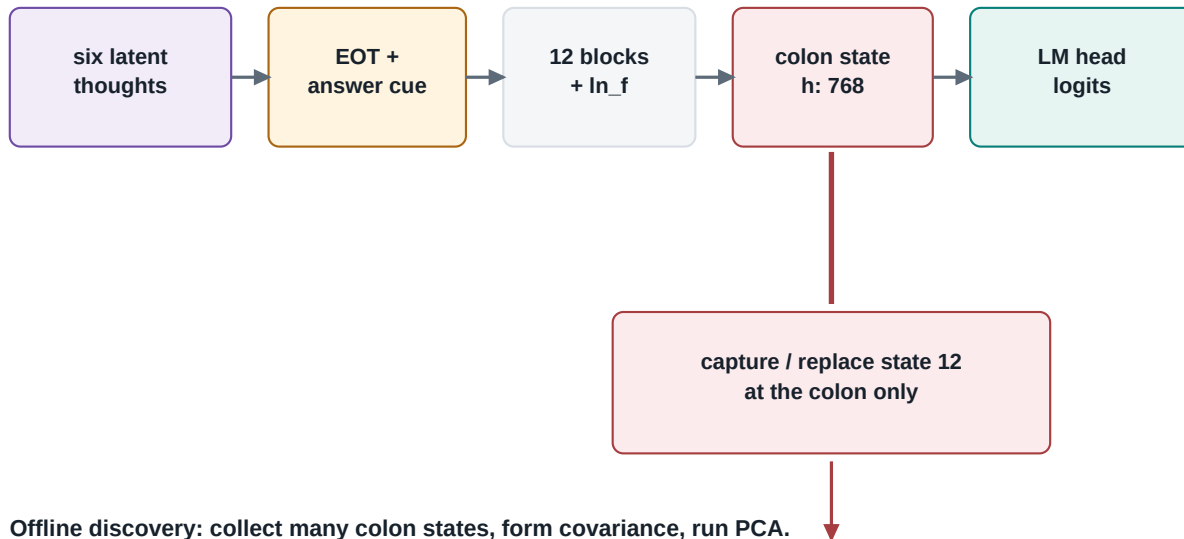
But A and U are not original CODI weights

Our PCA basis U is fitted from collected calibration states after training. The compressed matrix $A = WU$ and mean bias $b = mW^T$ are then computed offline. A production implementation would store A and b alongside the model and replace selected LM-head calls. The current repository result is a validated factorization and microbenchmark, not a fully deployed decoder head.

20. Our experimental access point

We wanted to know which parts of the student's answer-interface state contribute to correct generation. Therefore every example was sampled at one identical semantic location: after the cue colon has crossed all 12 blocks and final `ln_f`, immediately before the LM head.

WHERE OUR EXPERIMENT CONNECTS



Offline discovery: collect many colon states, form covariance, run PCA.

Causal confirmation: retain or remove selected PC bands, then perform real greedy answer generation.

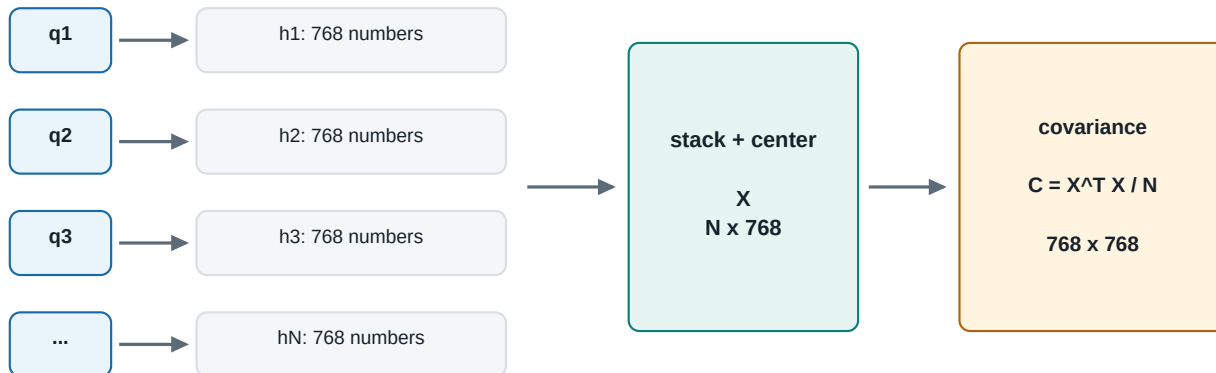
The hook does not modify earlier latent computations or model weights.

Why exact collection mattered

An earlier training-style collector reproduced only 89.06% of first tokens because question normalization, cue tokenization, left-padding position IDs, and generation details differed. Capturing the state with a hook in the actual generation path restored 100% first-token parity. This is a general lesson: a hidden state is defined by its full computational history, not only by its displayed text.

21. Building the 768 x 768 covariance matrix

FROM EXAMPLES TO PRINCIPAL DIRECTIONS



Eigenvectors of **C** are PCA directions **u0 ... u767**. Each is a 768-number unit vector.

They are ordered by variance, not by usefulness for answering.

The calibration set

We used 2,048 unique GSM8K training questions (seed 89), captured one 768-number student colon state per question, subtracted their mean **m**, and summarized co-variation with $C = X^T X / N$.

What covariance means

$C[a,b]$ is positive when coordinates **a** and **b** tend to rise together across examples, negative when one rises as the other falls, and near zero when their variation is unrelated. PCA diagonalizes this matrix into orthogonal axes ordered by how much variance each axis explains.

PCA is class-blind here. Correct and incorrect examples both contribute to the covariance. Accuracy enters later through causal generation tests.

22. What the 28 directions actually are

Each principal component is one unit vector in the original 768-dimensional space. It is a weighted pattern across all 768 coordinates. It is not one neuron, one token, one attention head, or one human-readable reasoning step.

```
u4  = [768 learned-from-data coefficients]
u5  = [768 learned-from-data coefficients]
...
u31 = [768 learned-from-data coefficients]

U_band = [u4 u5 ... u31] has shape 768 x 28
```

A toy three-dimensional analogy

Imagine points in a room. PC0 might describe mostly floor-to-ceiling movement, PC1 left-to-right, and PC2 a diagonal. Keeping two PCs means flattening every point onto their plane. In CODI, the room has 768 axes and the answer-bearing plane has 28 PCA axes.

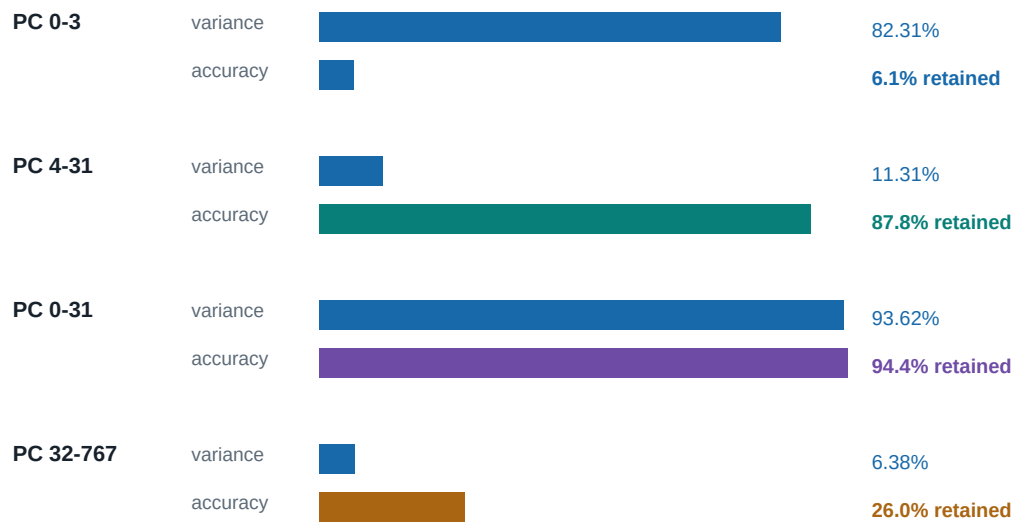
Why PC numbers begin at zero

PC0 has the most variance, PC1 the next most, and so on. Therefore PC4-31 contains 28 directions: $31 - 4 + 1 = 28$. The numbering says where they rank by variance, not how valuable they are.

The 28 directions were discovered in the population of colon states. CODI was not originally architected with a named 28-dimensional module.

23. The 28-direction result

THE SURPRISING BAND STRUCTURE



PC 4-31 carries little variance but most of the usable answer accuracy.

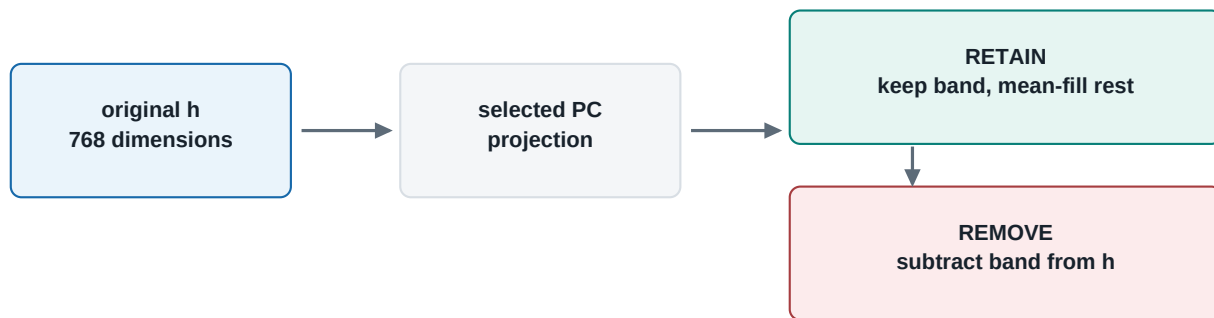
Baseline forced-cue exact match was 43.37% on all 1,319 GSM8K test questions. Keeping only PC 4-31 produced 38.06%, or 87.8% of baseline. Keeping only the four dominant-variance PCs produced 2.65%, or 6.1% of baseline.

The main conceptual result

Variance and answer contribution are nearly decoupled at this interface. Most state variance lives in PC0-3, yet those directions carry little numeric answer accuracy. A smaller mid-spectrum band carries most of what the final answer readout needs.

24. Why the result is causal, not just correlational

CAUSAL SUFFICIENCY AND NECESSITY



PC 4-31 retain: 38.06% exact match vs 43.37% baseline = 87.8% retained.

PC 4-31 remove: 12.89% exact match = a 30.48 percentage-point loss.

Random rank-28 subspaces retain only 6.5%-8.7% of baseline accuracy.

Conclusion is bounded to this checkpoint, task, semantic position, centering rule, and intervention family.

Sufficiency and necessity together

Retention asks whether the band alone is enough; removal asks whether the original state fails without it. PC4-31 passed both preregistered criteria. The exact-match results agreed closely with the cheap analytic first-token evaluator, supporting the linear state-12 calculations.

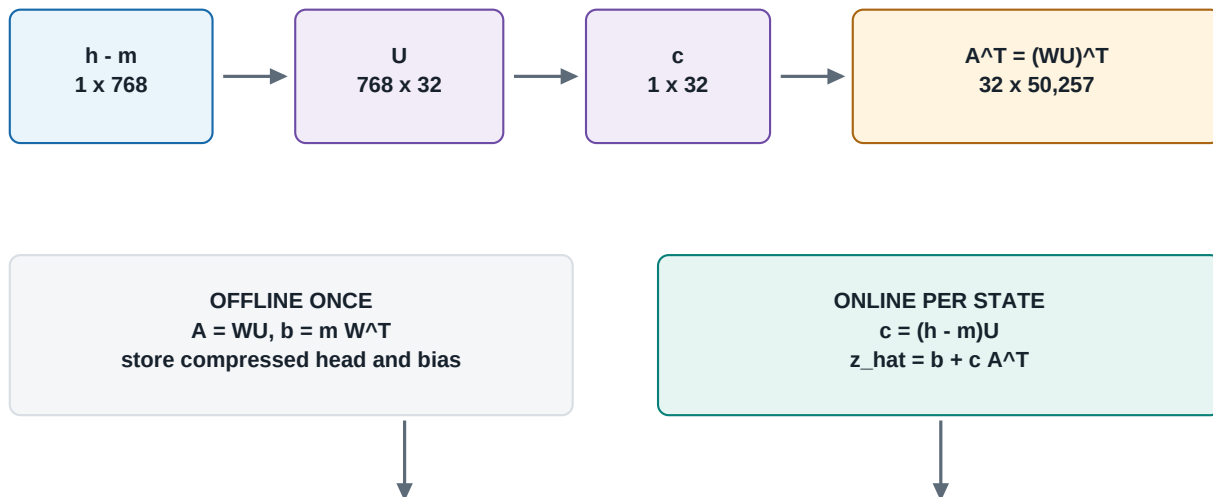
The important caveat

Keep-only interventions replace omitted dimensions with the calibration mean. Such states can be off the model's natural activation manifold. Donor/swap controls were weaker, and the strongest donor control was not repeated under full exact-match decoding. The conclusion is therefore strong but bounded, not universal.

25. From 28 scientific directions to a rank-32 readout

The 28-direction band is the scientific finding. For engineering, we add PC0-3 and use PC0-31, because four extra coordinates raise retained exact-match from 87.8% to 94.4%. This gives a cleaner accuracy/compute tradeoff.

TURNING THE FINDING INTO A SMALLER ANSWER READOUT



U contains the chosen PCA columns; it does not select literal rows or neurons from W.

A's row j tells how every vocabulary detector responds to PCA direction u_j .

The Transformer remains width 768. Only the final vocabulary scoring route is factorized.

How detectors change shape

W has one row per token and 768 columns. Multiplying W by U does not choose 32 rows. It expresses every token detector in the 32-direction coordinate system. $A = WU$ therefore keeps all 50,257 tokens but gives each only 32 coefficients.

$$W: 50,257 \times 768 \quad U: 768 \times 32 \quad A = WU: 50,257 \times 32$$

26. Operation counts and the measured speed

OPERATION COUNT AND MEASURED COMPONENT TIMING

Full LM head		38,597,376 MACs
Rank-32		1,632,800 MACs
MAC formula	$768 \times 32 + 32 \times 50,257 = 24,576 + 1,608,224$	
Arithmetic ratio	$38,597,376 / 1,632,800 = 23.64x$ fewer MACs	
T4 benchmark	full 1,327 us; rank-32 117 us = 11.3x faster LM-head microbenchmark	

This is not an 11.3x end-to-end CODI speedup: the twelve-block passes still dominate much of total generation.

What one MAC means

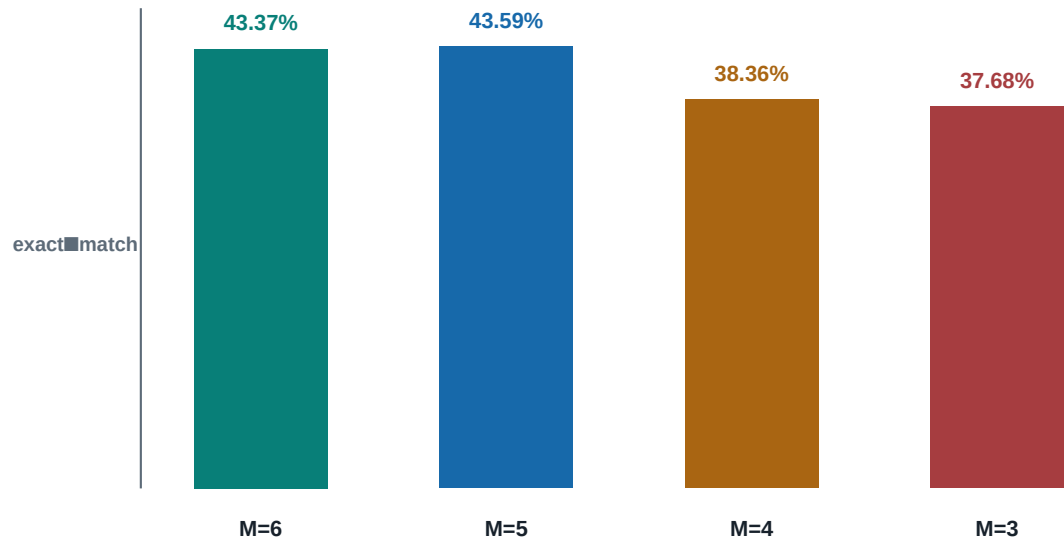
A multiply-accumulate multiplies two numbers and adds the product into a running sum. The full LM head needs one MAC for every state coordinate and token pair. The rank-32 path first computes 32 coordinates and then scores all tokens from those coordinates.

Why measured speedup is smaller than arithmetic reduction

GPU kernel launch overhead, reading matrices from memory, synchronization, and imperfect hardware utilization remain. Consequently, 23.6x fewer MACs produced an 11.3x component speedup in the isolated Tesla T4 benchmark.

27. Reducing the latent budget

A SECOND EFFICIENCY LEVER: FEWER LATENT PASSES



M=5 slightly exceeded M=6, but batch-32 T4 wall clock was flat; fixed and batched costs hid the saved pass.

Interpretation

The released checkpoint was trained with M=6. At inference, M=5 scored 43.59%, slightly above the M=6 baseline of 43.37%. M=4 and M=3 lost about five to six points. This suggests the sixth pass is redundant for this evaluation, while the fifth crosses a capacity boundary.

What was and was not faster

At batch 32 on a T4, measured wall clock was effectively flat across M because batched fixed costs dominated. In a latency-bound single-request deployment, skipping one of roughly sixteen sequential model stages could plausibly matter, but that end-to-end improvement was not directly measured here.

28. Relating the architecture to all our findings

WHAT EACH FINDING SAYS ABOUT THE ARCHITECTURE

Latent states are necessary	Inside six-pass loop The recurrent workspace matters as a whole
Values are decodable	Latent positions 1,3,5 Readable numeric shadows appear
Value injection failed	State 11 readout directions Readable is not the same as causally used
PC 4-31 carries accuracy	State 12 at answer colon Final answer interface is low-dimensional
Rank-32 head is faster	LM head only Vocabulary scoring can be factorized
M=5 preserves accuracy	Recurrent loop budget Last trained pass may be redundant

The findings localize different phenomena. They should not be collapsed into one story about a single 28-D reasoning algorithm.

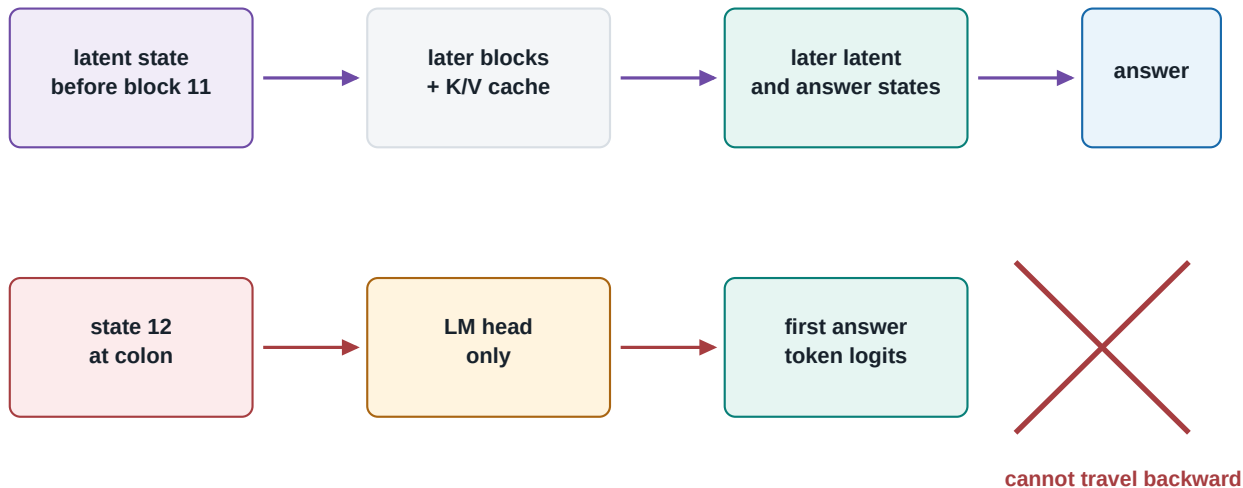
A layered interpretation

The latent loop is a distributed computational workspace. Some intermediate values can be decoded from it, but additive edits along their vocabulary readout directions did not causally change outcomes. Downstream, the final colon state contains a compact answer-bearing subspace that the LM head can read. These facts coexist because decodability, causal computation, and output accessibility are different properties.

Best current model: distributed computation inside the loop, compact answer access at the endpoint. This is a mechanism hypothesis consistent with the evidence, not proof of a human-readable 28-step algorithm.

29. Why intervention location changes the conclusion

CAUSAL BOUNDARIES: WHERE AN EDIT CAN TRAVEL



This is why endpoint geometry can identify the answer interface without proving where arithmetic was computed.

State 12 has no downstream Transformer block

An edit to the final state changes current logits, but it does not enter the KV cache for earlier latent positions and cannot revise earlier arithmetic. By contrast, an edit inside a latent pass before its final blocks can affect that position's K/V contribution and everything generated later.

The value-injection result

We added gold, offset, or random numeric-token readout directions at state 11 of selected latent value slots. Wrong-value injection left 99.45% of baseline-correct answers intact, identical to random; gold recovered only 3 of 248 baseline-wrong cases versus 1 for random. This supports the warning that a readable representation need not be the causally used code.

30. Training versus inference: do not mix the phases

Question	Training	Standard inference	Our experiment
Teacher trace present?	Yes, on teacher route	No	No
Student latent loop?	Yes, dynamically generated	Yes	Yes
Weights updated?	Yes	No	No
Gold answer supplied?	Yes, teacher forcing	No	No
Colon hidden KD?	Yes	No	No
PCA basis U?	No	No in original model	Yes, offline analysis/intervention
Rank-32 LM head?	No	Not integrated in released decoder	Microbenchmarked proposal

The most common conceptual errors

- Calling state 12 a thirteenth block.
- Treating each PCA direction as a neuron or vocabulary detector.
- Assuming the teacher runs during deployment.
- Assuming a faster LM head makes the entire model 11.3x faster.
- Equating a decoded value with a value the model causally uses.
- Claiming the 28 directions generalize beyond the tested checkpoint, endpoint, task, and intervention.

31. A worked end-to-end toy example

Suppose the question is 'Three bags contain four marbles each. How many marbles?' The real model uses 768-dimensional vectors; here we narrate the roles without pretending to know the exact internal arithmetic.

Stage	Toy event	Real architectural object
Encode	Words become vectors	Token + position embeddings, width 768
Question pass	Context is summarized	12 blocks; cache stores K/V
BOT	Silent reasoning begins	Special token at next position
z1-z6	Internal states evolve toward an answer	Projector + six cached GPT-2 passes
EOT + cue	Model switches to answer mode	Special token plus 'The answer is:'
Colon state	Answer-ready vector h	State 12 after block 11 + ln_f
LM head	' 12' receives top score	$h W^T \rightarrow 50,257$ logits
Decode	12 is emitted	Greedy argmax; token fed back

With the rank-32 proposal

Only the final scoring line changes: compute $c = (h-m)U$, then $b+cA^T$. All question processing, latent passes, attention, MLPs, cache updates, and answer-token recurrence remain full 768-dimensional CODI computation.

Low-rank readout compresses how the answer-ready state is read, not how the answer-ready state is produced.

32. Equations and shapes you should be able to reconstruct

A COMPACT SYMBOL MAP

h	final state at one position	1×768
W	stored vocabulary detector matrix	$50,257 \times 768$
m	calibration mean state	1×768
U	selected PCA basis, usually PC 0-31	768×32
c	coordinates of centered h in U	1×32
A = WU	offline compressed vocabulary head	$50,257 \times 32$
z	vocabulary scores	$1 \times 50,257$

Normal readout

$$h W^T = z$$

$$(1 \times 768)(768 \times 50,257) = 1 \times 50,257$$

Projection and reconstruction

$$c = (h - m) U \quad \# 1 \times 32$$

$$h_{\text{projected}} = m + c U^T \quad \# 1 \times 768$$

$$z_{\text{projected}} = h_{\text{projected}} W^T \quad \# 1 \times 50,257$$

Reassociated low-rank readout

$$A = WU \quad \# \text{ offline: } 50,257 \times 32$$

$$b = m W^T \quad \# \text{ offline: } 1 \times 50,257$$

$$z_{\text{hat}} = b + c A^T \quad \# \text{ online: } 1 \times 50,257$$

Reassociating multiplication is exact for the projected state. The approximation comes only from discarding the dimensions outside U.

33. What remains to be validated

The experiments establish a real low-dimensional answer interface, but an engineering deployment needs more work.

- Integrate the rank-32 head into the full autoregressive decoder and apply it at every desired answer position.
- Measure batch-1, batch-32, and realistic server latency end to end, including data movement and kernel overhead.
- Test whether one fixed basis works for later answer tokens, other prompts, other datasets, and other checkpoints.
- Evaluate top-k/top-p sampling, calibration, log-likelihood, and nonnumeric text - not only greedy numeric exact match.
- Train with subspace-aware regularization or distillation if the goal is to recover lost accuracy rather than accept 94.4% retention.
- Use held-out bases and on-manifold controls to reduce concerns about mean-filled synthetic states.

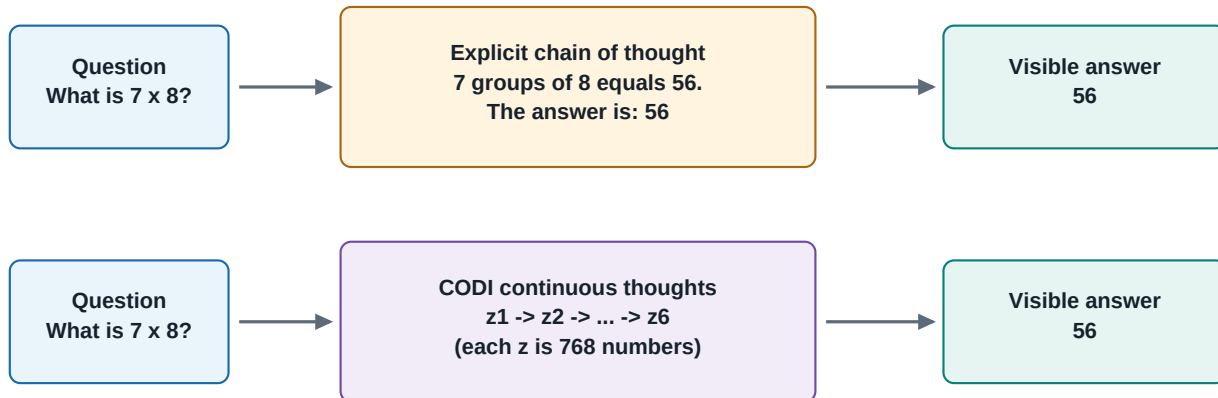
A sensible mitigation ladder

Mitigation	Cost	Expected benefit
Increase rank 32 -> 64	More LM-head MACs	Likely recover more logits/accuracy
Keep full head for uncertain states	Adaptive branch	Protect difficult examples
Fine-tune with projected head	New training run	Teach model to place answer signal inside U
Use low-rank only for first numeric token	Limited scope	Match the position where basis was validated

34. Final mental model

CODI is a GPT-2 causal decoder used recurrently. It learns a compact student route by making its answer-cue representation resemble that of an explicit chain-of-thought teacher route while both roles share weights.

TWO WAYS TO REASON



Training uses explicit reasoning as a teacher signal; inference uses the compact student route.

Remember the architecture as five verbs

- **Embed:** turn question tokens into 768-number vectors.
- **Attend:** pass information through 12 causal Transformer blocks while building a KV cache.
- **Recur:** project and feed back six continuous thoughts.
- **Switch:** insert EOT and the answer cue to enter answer mode.
- **Read:** map the colon state through the LM head and decode vocabulary tokens.

Our finding adds one sentence: at the tested answer interface, most numeric exact-match accuracy is accessible through PC 4-31, and PC 0-31 gives a practical rank-32 readout candidate.

35. Glossary

Term	Meaning in this guide
BOT / EOT	Learned markers for beginning/end of continuous reasoning
Continuous thought	A learned 768-number vector used directly as the next input embedding
Hidden state	The representation at one sequence position and one depth
State 12	Final In_f output after GPT-2 block 11
KV cache	Stored attention keys and values for earlier positions at all layers
LM head	Stored linear vocabulary detector bank mapping 768 -> vocabulary logits
Logit	Unnormalized score for one vocabulary token
Teacher forcing	Supplying gold previous tokens during training
Self-distillation	Shared model plays teacher/student roles with a representation-matching loss
PCA direction	An orthogonal axis of population variation, expressed across all 768 coordinates
Retention	Keep selected projection and replace complement with calibration mean
Removal	Subtract selected projection from the original state
MAC	One multiply-accumulate operation

One-sentence warning

A direction can be statistically visible, linearly decodable, causally necessary at one endpoint, and still not be a standalone human-readable algorithm. Always specify the location, operation, comparison, and behavioral metric.

36. Sources, provenance, and scope

Primary CODI sources

- Shen et al., 'CODI: Compressing Chain-of-Thought into Continuous Space via Self-Distillation,' EMNLP 2025. ACL Anthology PDF: <https://aclanthology.org/2025.emnlp-main.36.pdf>
- ArXiv record: <https://arxiv.org/abs/2502.21074>
- Official implementation: <https://github.com/zhenyi4/codi>, source revision 2c2314662c63e9f482ebc46614ffe9af17a241e5
- Released GPT-2 checkpoint used by this repository: zen-E/CODI-gpt2, checkpoint revision fd641b3d3edc59e4f534b55588e906588c9e36bb.

Local verified experiment record

- docs/RESEARCH_CONTEXT_LEDGER.md, especially sections 40, 58, and 59.
- docs/OFFICIAL_CODI_ENDPOINT_BAND_CONFIRMATION.md and docs/OFFICIAL_CODI_ENDPOINT_MARGIN_GEOMETRY.md.
- src/models/official_codi.py and src/data/official_codi_training.py for the source-faithful runtime and sequence contract.
- docs/OFFICIAL_CODI_VALUE_INJECTION.md for the latent value-injection result.

Scope statement

Official architecture/training descriptions come from the paper and public code. Exact-match, PCA-band, value-injection, latent-budget, and LM-head timing results come from this repository's verified experiment ledger. The low-rank head is an inference opportunity validated algebraically and by a component microbenchmark; it is not yet an integrated production speedup.

Bounded conclusion: official CODI GPT-2, GSM8K, six-thought checkpoint, forced answer cue, state 12 at the colon, linear PCA subspaces, greedy numeric exact match.